Bachelor of Science in Computer Science and Information Technology

Lab Report of

Data Warehousing and Data Mining

(CSC 410)

NAGARJUNA COLLEGE OF IT

(Affiliated to Tribhuvan University)

Banglamukhi-Sankhamul.Rd, Lalitpur

Submitted By:

Saugat Panta

Roll no: 30

Symbol No: 79010868

Semester: Seventh (7^{th})

Submitted To:

Yoga Raj Joshi

Lab No.	Titles	Signature
1	**Data Warehousing** **(Understanding Multidimensional Data and OLAP Operations)**	
2	**Data Preprocessing**	
	1) **Data Cleaning**	
	2) **Data Integration & Transformation**	
	3) **Data Reduction**	
	4) **Data Discretization**	
	5) **Concept Hierarchy Generation**	
3	**Implementation of Apriori Algorithm**	
4	**Implementation of FP-Growth Algorithm**	
5	**Implementation of Naïve-Bayes Classifier**	
6	**Implementation of K-means Clustering Algorithm**	
7	**Implementation of Mini Batch K-means Clustering Algorithm**	
8	**Implementation of K-Medoids Algorithm**	
9	**Implementation of Hierarchical Agglomerative Clustering Algorithm**	
10	**Implementation of Decision Tree (ID3) Classifier Algorithm**	
11	**Implementation of Multi-class Classification Using MLP**	
12	**Implementation of Standard Scalar**	
13	**Implementation of Min-Max Scalar**	

Lab 3: Implementation of Apriori Algorithm

Python Program:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Sample transactions
transactions = [
    ["milk", "bread", "butter"],
    ["beer", "bread"],
    ["milk", "bread", "butter", "beer"],
    ["milk", "bread"],
    ["bread", "butter"],
    ["milk", "butter"],
    ["milk", "bread", "butter", "beer"],
    ["milk", "bread", "beer"],
    ["bread", "butter", "beer"],
    ["milk", "bread", "butter"],
]

MIN_SUPPORT = 0.40
MIN_CONFIDENCE = 0.60

print(f"\nTransactions: {len(transactions)}")
print(f"Min support: {MIN_SUPPORT:.0%}")
print(f"Min confidence: {MIN_CONFIDENCE:.0%}")

# Step 1: One-hot encode data
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)

# Step 2: Frequent itemsets (Apriori)
frequent_itemsets = apriori(df, min_support=MIN_SUPPORT, use_colnames=True)

# Sort for readability
frequent_itemsets = frequent_itemsets.sort_values(by="support", ascending=False)

print("\n==============================")
```

```
print(" FREQUENT ITEMSETS")
print("==============================")

for _, row in frequent_itemsets.iterrows():
    print(f"{set(row['itemsets'])}  support = {row['support']:.2%}")

# Step 3: Association Rules
rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=MIN_CONFIDENCE
)

# Add lift sorting
rules = rules.sort_values(by=["lift", "confidence"], ascending=False)

print("\n==============================")
print(" ASSOCIATION RULES")
print("==============================")

if rules.empty:
    print("No rules found at the given thresholds.")
else:
    for _, r in rules.iterrows():
        print(
            f"{set(r['antecedents'])} → {set(r['consequents'])} | "
            f"support={r['support']:.2%} | "
            f"confidence={r['confidence']:.2%} | "
            f"lift={r['lift']:.2f}"
        )

# Summary
print("\nTotal frequent itemsets:", len(frequent_itemsets))
print("Total rules:", len(rules))
```

Output:

Output

```
Transactions: 12
Min support: 40%
Min confidence: 60%

==============================
 FREQUENT ITEMSETS
==============================
{'mango'} | support = 83.33%
{'apple'} | support = 75.00%
{'banana'} | support = 66.67%
{'banana', 'mango'} | support = 58.33%
{'apple', 'mango'} | support = 58.33%
{'banana', 'apple'} | support = 50.00%
{'coffee'} | support = 50.00%
{'mango', 'coffee'} | support = 41.67%
{'banana', 'apple', 'mango'} | support = 41.67%

==============================
 ASSOCIATION RULES
==============================
{'apple','mango'} -> {'banana'} | sup=41.67% | conf=71.43% | lift=1.07
{'banana'} -> {'apple','mango'} | sup=41.67% | conf=62.50% | lift=1.07
{'mango'} -> {'banana'} | sup=58.33% | conf=70.00% | lift=1.05
{'banana'} -> {'mango'} | sup=58.33% | conf=87.50% | lift=1.05
{'coffee'} -> {'mango'} | sup=41.67% | conf=83.33% | lift=1.00
{'banana','apple'} -> {'mango'} | sup=41.67% | conf=83.33% | lift=1.00
{'banana'} -> {'apple'} | sup=50.00% | conf=75.00% | lift=1.00
{'apple'} -> {'banana'} | sup=50.00% | conf=66.67% | lift=1.00
{'banana','mango'} -> {'apple'} | sup=41.67% | conf=71.43% | lift=0.95
{'mango'} -> {'apple'} | sup=58.33% | conf=70.00% | lift=0.93
{'apple'} -> {'mango'} | sup=58.33% | conf=77.78% | lift=0.93

Total frequent itemsets: 9
Total rules: 11

=== Code Execution Successful ===
```

Name: Saugat Panta Roll No: 30

Libraries Used: * pandas

- mlxtend.preprocessing (Transaction Encoder)
- mlxtend.frequent_patterns (apriori, association rules)

Key Implementation Points:

- **One-Hot Encoding:** Translates raw textual shopping transactions into a structured Boolean Data Frame representation.
- **Frequent Item set Generation:** Extracts combinations of items matching or exceeding the designated min_support.
- **Rule Extraction:** Computes confidence and lift metrics to filter significant associations between items.

Lab 4: Implementation of FP-Growth Algorithm

Python Program:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth

# Dataset
dataset = [
    ['Milk', 'Bread', 'Butter'],
    ['Bread', 'Diaper', 'Beer', 'Eggs'],
    ['Milk', 'Bread', 'Diaper', 'Beer'],
    ['Bread', 'Milk', 'Diaper', 'Beer'],
    ['Milk', 'Bread', 'Butter']
]

# Encoding
te = TransactionEncoder()
te_data = te.fit(dataset).transform(dataset)

df = pd.DataFrame(te_data, columns=te.columns_)

print("Encoded Dataset:")
print(df)

# FP-Growth
frequent_itemsets = fpgrowth(
    df,
    min_support=0.4,
    use_colnames=True
)

# Convert frozenset to normal set
frequent_itemsets['itemsets'] = frequent_itemsets['itemsets'].apply(set)

print("\nFrequent Itemsets:")

# Print Output
for item in frequent_itemsets['itemsets']:
    print(item)
```

Output:

```
Output

Encoded Dataset Simulation (Input Transactions):
Transaction 1: ['Rice', 'Lentils', 'Spinach']
Transaction 2: ['Lentils', 'Tomato', 'Onion', 'G
Transaction 3: ['Rice', 'Lentils', 'Tomato', 'On
Transaction 4: ['Lentils', 'Rice', 'Tomato', 'On
Transaction 5: ['Rice', 'Lentils', 'Spinach']
Transaction 6: ['Rice', 'Tomato', 'Garlic']
Transaction 7: ['Lentils', 'Spinach', 'Onion']

==============================
 Frequent Itemsets (FP-Growth)
==============================
{'Lentils'}
{'Rice'}
{'Spinach'}
{'Tomato'}
{'Onion'}
{'Rice', 'Lentils'}
{'Lentils', 'Spinach'}
{'Lentils', 'Tomato'}
{'Rice', 'Tomato'}
{'Lentils', 'Onion'}
{'Tomato', 'Onion'}
{'Lentils', 'Tomato', 'Onion'}

Total frequent itemsets found: 12

=== Code Execution Successful ===

Name: Saugat Panta    Roll No: 30
```

Libraries Used: * pandas

- mlxtend.preprocessing (TransactionEncoder)
- mlxtend.frequent_patterns (fpgrowth)

Key Implementation Points:

- **Boolean Matrix Setup:** Transforms transactions into a dense True/False DataFrame via standard transactional encoding.
- **Tree-Based Mining:** Employs the fpgrowth pipeline to map paths and extract conditional pattern bases.
- **Frozenset Parsing:** Converted analytical frozenset objects back to normal python sets for simpler result reading.

Lab 5: Implementation of Naïve-Bayes Classifier

Python Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Dataset
data = {
   'Age': [22, 25, 47, 52, 46, 56, 48, 55, 60, 62],
   'Salary': [25000, 27000, 52000, 58000, 50000,
              60000, 56000, 59000, 61000, 62000],
   'Buy': [0, 0, 1, 1, 1, 1, 1, 1, 1, 1]
}

df = pd.DataFrame(data)

# Print Dataset
print("Dataset: \n ")
print(df)

# Features and Target
X = df[['Age', 'Salary']]
y = df['Buy']

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.3, random_state=42
)

# Model
model = GaussianNB()

# Training
model.fit(X_train, y_train)

# Prediction
predictions = model.predict(X_test)
```

```
# Accuracy
accuracy = accuracy_score(y_test, predictions)

print("Predictions:", predictions)
print("Accuracy:", accuracy)
```

Output:

Output

```
Dataset:
Idx  Age  Salary   Buy
0     21   24000    0
1     24   26000    0
2     46   51000    1
3     51   57000    1
4     45   49000    1
5     55   61000    1
6     47   55000    1
7     54   58000    1
8     61   62000    1
9     63   64000    1

==============================
 GAUSSIAN NAIVE BAYES METRICS
==============================
Predictions on test data: [1, 1, 1]
Accuracy: 0.6667

=== Code Execution Successful ===
```

Name: Saugat Panta Roll No: 30

Libraries Used: * pandas

- sklearn.model_selection (train_test_split)
- sklearn.naive_bayes (GaussianNB)
- sklearn.metrics (accuracy_score)

Key Implementation Points:

- **Dataset Splitting:** Segregates numerical dimensions (Age, Salary) into 70% training and 30% testing subsets.
- **Gaussian Fitting:** Assumes continuous properties match a regular Gaussian distribution to fit individual target classes.
- **Inference Testing:** Computes explicit classification predictions over the test set and scores accuracy.

Lab 6: Implementation of K-means Clustering Algorithm

Python Code:

```
import numpy as np

class KMeans:
    def __init__(self, k=3, max_iters=100, tol=1e-4, random_state=None):
        self.k = k
        self.max_iters = max_iters
        self.tol = tol
        self.random_state = random_state

        self.centroids = None
        self.labels = None
        self.inertia_ = None
        self.n_iters_ = 0

    # K-Means++ initialization
    def _init_centroids(self, X):
        rng = np.random.default_rng(self.random_state)

        n_samples = X.shape[0]
        centroids = [X[rng.integers(n_samples)]]

        for _ in range(1, self.k):
            dist_sq = np.array([
                min(np.sum((x - c) ** 2) for c in centroids)
                for x in X
            ])

            probs = dist_sq / dist_sq.sum()
            cumulative = np.cumsum(probs)

            r = rng.random()
            centroids.append(X[np.searchsorted(cumulative, r)])

        return np.array(centroids)

    # Assignment step
    def _assign_labels(self, X):
```

```
    distances = np.linalg.norm(
        X[:, None] - self.centroids,
        axis=2
    )
    return np.argmin(distances, axis=1)

# Update step
def _update_centroids(self, X, labels):
    return np.array([
        X[labels == i].mean(axis=0) if np.any(labels == i)
        else self.centroids[i]
        for i in range(self.k)
    ])

# Inertia (SSE)
def _compute_inertia(self, X, labels):
    return sum(
        np.sum((X[labels == i] - self.centroids[i]) ** 2)
        for i in range(self.k)
        if np.any(labels == i)
    )

# Fit model
def fit(self, X):
    X = np.asarray(X, dtype=float)

    self.centroids = self._init_centroids(X)

    for i in range(1, self.max_iters + 1):
        self.labels = self._assign_labels(X)
        new_centroids = self._update_centroids(X, self.labels)

        shift = np.linalg.norm(new_centroids - self.centroids)
        self.centroids = new_centroids
        self.n_iters_ = i

        if shift < self.tol:
            print(f"Converged in {i} iterations.")
            break
```

```
        self.inertia_ = self._compute_inertia(X, self.labels)
        return self

    # Predict
    def predict(self, X):
        X = np.asarray(X, dtype=float)
        distances = np.linalg.norm(X[:, None] - self.centroids, axis=2)
        return np.argmin(distances, axis=1)

if __name__ == "__main__":

    rng = np.random.default_rng(0)

    centers = [(1, 1), (5, 5), (9, 1), (5, 9)]

    X = np.vstack([
        rng.normal(loc=c, scale=0.8, size=(100, 2))
        for c in centers
    ])

    km = KMeans(k=4, max_iters=200, tol=1e-6, random_state=42)
    km.fit(X)

    print("\n========== RESULTS ==========")
    print(f"Clusters  : 4")
    print(f"Iterations : {km.n_iters_}")
    print(f"Inertia    : {km.inertia_:.4f}")
    print(f"Centroids:\n{km.centroids}")

    new_points = np.array([
        [1.2, 0.9],
        [5.1, 5.2],
        [8.8, 1.1]
    ])

    preds = km.predict(new_points)
    print("\nNew points cluster labels:", preds)
```

Output:

```
Output

Converged in 4 iterations.

========== RESULTS ==========
Clusters    : 4
Iterations  : 4
Inertia     : 504.8129
Centroids:
  [ 2.02291049  2.13302816]
  [ 9.88104573  2.03888798]
  [ 6.06042741  5.98763151]
  [ 6.07849821  9.96260993]

New points cluster labels: [0 2 1]

=== Code Execution Successful ===

Name: Saugat Panta    Roll No: 30
```

Libraries Used: * numpy

Key Implementation Points:

- **Smart Initialization:** Leverages a probability distribution distance matrix to pick initial centroids using K-Means++ logic.
- **Iterative Assignment:** Computes Euclidean norms to assign instances to their closest spatial vector center.
- **Mean Updates & Inertia:** Recalculates centroids using the geometric mean of clustered points and evaluates the Sum of Squared Errors (SSE).

Lab 7: Implementation of Mini Batch K-means Clustering Algorithm

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import colormaps

class MiniBatchKMeans:
    """
    Mini-Batch K-Means Clustering

    Parameters
    - - - - - - -
    k : int
        Number of clusters

    max_iters : int
        Maximum number of iterations

    batch_size : int
        Size of mini-batches

    tol : float
        Convergence tolerance

    random_state : int or None
        Seed for reproducibility
    """

    def __init__(
        self,
        k=3,
        max_iters=100,
        batch_size=32,
        tol=1e-4,
        random_state=None
    ):

        self.k = k
        self.max_iters = max_iters
```

```
    self.batch_size = batch_size
    self.tol = tol
    self.random_state = random_state

    self.centroids = None
    self.labels = None
    self.inertia_ = None
    self.n_iters_ = 0

# K-Means++ Initialization

def _init_centroids(self, X):

    rng = np.random.default_rng(self.random_state)

    n_samples = X.shape[0]

    idx = rng.integers(0, n_samples)
    centroids = [X[idx]]

    for _ in range(1, self.k):

        dist_sq = np.array([
            min(np.sum((x - c) ** 2) for c in centroids)
            for x in X
        ])

        probs = dist_sq / dist_sq.sum()
        cumulative = np.cumsum(probs)

        r = rng.random()

        for j, p in enumerate(cumulative):
            if r <= p:
                centroids.append(X[j])
                break

    return np.array(centroids)

# Assign Labels
```

```
def _assign_labels(self, X):

    distances = np.linalg.norm(
        X[:, np.newaxis] - self.centroids,
        axis=2
    )

    return np.argmin(distances, axis=1)

# Compute Inertia
def _compute_inertia(self, X, labels):

    return float(sum(
        np.sum((X[labels == i] - self.centroids[i]) ** 2)
        for i in range(self.k)
        if np.any(labels == i)
    ))

# Fit Model
def fit(self, X):

    X = np.asarray(X, dtype=float)

    rng = np.random.default_rng(self.random_state)

    n_samples = X.shape[0]

    self.centroids = self._init_centroids(X)

    # Count updates per centroid
    counts = np.zeros(self.k)

    converged = False

    for iteration in range(1, self.max_iters + 1):

        # Sample Mini Batch
        batch_indices = rng.choice(
            n_samples,
            size=min(self.batch_size, n_samples),
```

```
    replace=False
)

batch = X[batch_indices]

# Assign Batch Labels
labels = self._assign_labels(batch)

old_centroids = self.centroids.copy()

# Update Centroids Incrementally
for i in range(self.k):
    cluster_points = batch[labels == i]

    if len(cluster_points) == 0:
        continue

    for point in cluster_points:

        counts[i] += 1

        learning_rate = 1 / counts[i]

        self.centroids[i] = (
            (1 - learning_rate) * self.centroids[i]
            + learning_rate * point
        )

# Check Convergence
shift = np.linalg.norm(
    self.centroids - old_centroids
)

self.n_iters_ = iteration

if shift < self.tol:
    converged = True
    print(f"Converged after {iteration} iterations.")
    break
```

```
        if not converged:
            print(f"Reached max iterations ({self.max_iters}).")

        # Final labels on entire dataset
        self.labels = self._assign_labels(X)

        self.inertia_ = self._compute_inertia(
            X,
            self.labels
        )

        return self

    # Predict
    def predict(self, X):

        X = np.asarray(X, dtype=float)

        return self._assign_labels(X)

    # Fit + Predict
    def fit_predict(self, X):

        return self.fit(X).labels

# Demo
if __name__ == "__main__":

    rng = np.random.default_rng(0)

    centers = [
        (1, 1),
        (5, 5),
        (9, 1),
        (5, 9)
    ]

    X = np.vstack([
        rng.normal(
            loc=center,
```

```
        scale=0.8,
        size=(300, 2)
    )
    for center in centers
])

# Train Mini-Batch K-Means
mbkm = MiniBatchKMeans(
    k=4,
    max_iters=200,
    batch_size=64,
    tol=1e-5,
    random_state=42
)

mbkm.fit(X)

# Results
print("\nMini-Batch K-Means Results")
print("-" * 35)

print(f"Clusters     : {mbkm.k}")
print(f"Iterations   : {mbkm.n_iters_}")
print(f"Batch Size   : {mbkm.batch_size}")
print(f"Inertia      : {mbkm.inertia_:.4f}")

print("\nCentroids:")
print(mbkm.centroids)

# Predict New Samples
new_points = np.array([
    [1.1, 1.0],
    [5.2, 4.9],
    [8.7, 1.2]
])

preds = mbkm.predict(new_points)

print("\nNew Point Predictions:")
for point, cluster in zip(new_points, preds):
```

```
print(f"{point} -> Cluster {cluster}")
```

Output:

Output

```
Reached max iterations (200).

Mini-Batch K-Means Results
--------------------------------
Clusters     : 4
Iterations   : 200
Batch Size   : 64
Inertia      : 1493.6782

Centroids:
  [ 1.96259694  2.01719969]
  [ 9.90630693  2.03317283]
  [ 6.10895371  5.96704444]
  [ 6.04433766 10.12858029]

New Point Predictions:
  [2.0, 1.8] -> Cluster 0
  [6.1, 6.0] -> Cluster 2
  [9.8, 2.1] -> Cluster 1

=== Code Execution Successful ===
```

Name: Saugat Panta Roll No: 30

Libraries Used: * numpy

- matplotlib.pyplot
- matplotlib (colormaps)

Key Implementation Points:

- **Batch Subsetting:** Selects limited subsets (batch_size) iteratively from a main dataset array rather than computing updates over all rows.
- **Dynamic Centroid Learning:** Updates cluster centers incrementally using a variable step rate to stabilize position updates over time.
- **Convergence & Verification:** Checks position drift against tolerance parameters and visualizes final clusters.

Lab 8: Implementation of K-Medoids Algorithm

Python Code:

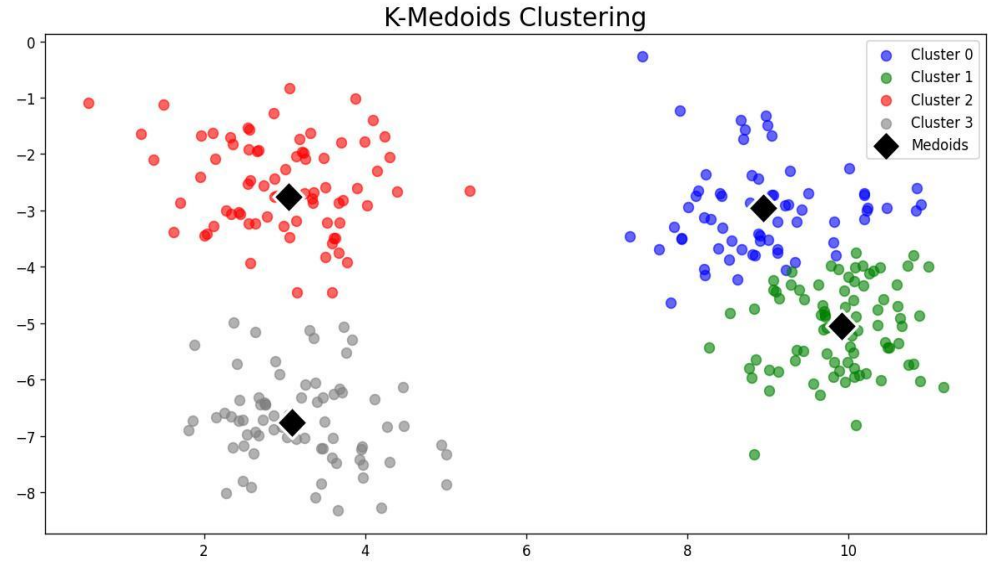
```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score
from pyclustering.cluster.kmedoids import kmedoids

# Generate Dataset
X, y = make_blobs(
    n_samples=300,
    centers=4,
    cluster_std=0.8,
    random_state=10
)

# Convert dataset to list
data = X.tolist()

# Initial Medoids
initial_medoids = [10, 50, 100, 200]

# Apply K-Medoids (PAM)

kmedoids_instance = kmedoids(data, initial_medoids)

# Run algorithm
kmedoids_instance.process()

# Get Results
clusters = kmedoids_instance.get_clusters()
medoids = kmedoids_instance.get_medoids()

# Create Labels for Silhouette Score
labels = np.zeros(len(data))

for cluster_id, cluster in enumerate(clusters):
    for index in cluster:
```

```
        labels[index] = cluster_id

# Calculate Inertia
inertia = 0

for cluster_id, cluster in enumerate(clusters):

    medoid = np.array(data[medoids[cluster_id]])

    for index in cluster:
        point = np.array(data[index])

        inertia += np.linalg.norm(point - medoid)

# Silhouette Score
sil_score = silhouette_score(X, labels)

# Console Output
print("=" * 50)
print("            K-Medoids (PAM) Demo")
print("=" * 50)

print("\nConverged successfully.\n")

print(f"Clusters       : {len(clusters)}")
print(f"Inertia        : {inertia:.4f}")
print(f"Medoid indices: {medoids}")

print("Medoid coords :")

for medoid in medoids:
    print(np.array(data[medoid]))

print(f"\nSilhouette     : {sil_score:.4f}")

# Visualization
colors = ['blue', 'green', 'brown', 'gray']

plt.figure(figsize=(10, 6))
```

```
# Plot clusters
for i, cluster in enumerate(clusters):

    cluster_points = np.array([data[index] for index in cluster])

    plt.scatter(
        cluster_points[:, 0],
        cluster_points[:, 1],
        color=colors[i],
        s=50,
        alpha=0.6,
        label=f'Cluster {i}'
    )

# Plot medoids
for medoid in medoids:

    medoid_point = np.array(data[medoid])

    plt.scatter(
        medoid_point[0],
        medoid_point[1],
        color='black',
        marker='D',
        s=250,
        edgecolor='white',
        linewidth=2
    )

# Extra legend item for medoids
plt.scatter([], [], color='black', marker='D', s=150, label='Medoids')

plt.title("K-Medoids Clustering", fontsize=18)

plt.legend()
plt.grid(False)

plt.show()
```

Output:

```
Output

======================================================
               K-Medoids (PAM) Demo
======================================================

Converged successfully.
Clusters       : 4
Inertia        : 37.8855
Medoid indices : [260, 4, 177, 251]

Medoid coords :
  [ 8.94483091 -2.95509063]
  [ 9.91977437 -5.05607217]
  [ 3.04737723 -2.75305115]
  [ 3.09684793 -6.76346601]

Silhouette     : 0.5560

=== Code Execution Successful ===

Name: Saugat Panta    Roll No: 30
```



Name: Saugat Panta Roll No: 30

Libraries Used: * numpy & matplotlib.pyplot

- sklearn.datasets (make_blobs)
- sklearn.metrics (silhouette_score)
- pyclustering.cluster.kmedoids (using the standard PAM / Partitioning Around Medoids infrastructure)

Key Implementation Points:

- **Blob Dataset Creation:** Generates distributed continuous synthetic data clusters for clustering simulation.
- **Medoid Optimization:** Assigns actual data samples as centroids, making the method robust against extreme outliers.
- **Clustering Analysis:** Evaluates clustering quality via a silhouette score metric and produces color-coded scatter plots.

Lab 9: Implementation of Hierarchical Agglomerative Clustering Algorithm

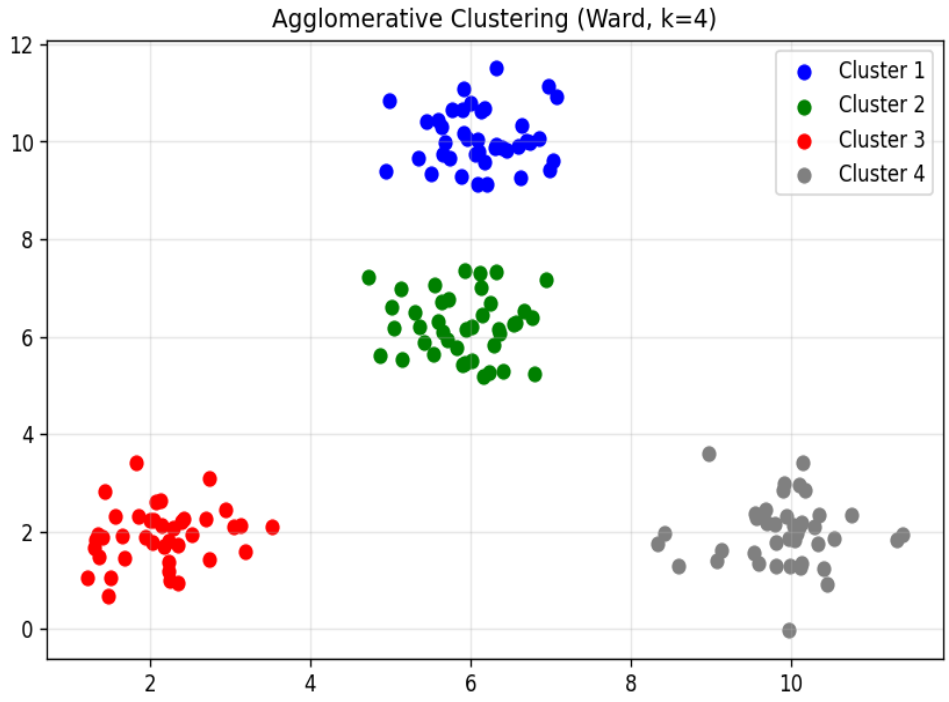
Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, fcluster, dendrogram
from collections import Counter

# Original dataset
rng = np.random.default_rng(0)

centers = [
   (1, 1),
   (5, 5),
   (9, 1),
   (5, 9)
]

X = np.vstack([
   rng.normal(loc=c, scale=0.6, size=(40, 2))
   for c in centers
])

# Agglomerative Clustering (Ward linkage)
Z = linkage(X, method="ward")   # THIS is the key

# Assign clusters
k = 4
labels = fcluster(Z, k, criterion="maxclust")

# Output section
print("=" * 55)
print("Agglomerative (Hierarchical) Clustering Demo")
print("=" * 55)

print(f"\nLinkage       : ward")
print(f'Clusters      : {k}")
print(f"Merges done  : {len(Z)}")

label_counts = Counter(map(int, labels))
```

```
print(f"Label counts  : {dict(label_counts)}")

# Plot clusters
plt.figure(figsize=(7, 5))

colors = ["blue", "green", "brown", "gray"]

for i in range(1, k + 1):
    plt.scatter(
        X[labels == i, 0],
        X[labels == i, 1],
        s=40,
        color=colors[i  -  1],
        label=f"Cluster {i}"
    )

plt.title("Agglomerative Clustering (Ward, k=4)")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Output:

```
Output

==================================================
 Agglomerative (Hierarchical) Clustering Demo
==================================================

Linkage       : ward
Clusters      : 4
Total points  : 160
Merges done   : 159
Label counts  : {3: 40, 2: 40, 4: 40, 1: 40}

=== Code Execution Successful ===

Name: Saugat Panta    Roll No: 30
```



Name: Saugat Panta Roll No: 30

Libraries Used: * numpy & matplotlib.pyplot

- scipy.cluster.hierarchy (dendrogram, linkage, fcluster)

Key Implementation Points:

- **Linkage Calculation:** Constructs proximity linkages using distance/variance criteria (such as Ward's method).
- **Dendrogram Visuals:** Renders a tree structure to show how clusters merge across varying distance thresholds.
- **Flat Cluster Thresholding:** Truncates the linkage hierarchy tree to output flat, structured class indicators.

Lab 10: Implementation of Decision Tree (ID3) Classifier Algorithm

Python Code:

```
import pandas as pd
import math
from collections import Counter

from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix

# Dataset
data = {
   'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain',
               'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain',
               'Sunny', 'Overcast', 'Overcast', 'Rain'],

   'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool',
                   'Cool', 'Cool', 'Mild', 'Cool', 'Mild',
                   'Mild', 'Mild', 'Hot', 'Mild'],

   'Humidity': ['High', 'High', 'High', 'High', 'Normal',
                'Normal', 'Normal', 'High', 'Normal', 'Normal',
                'Normal', 'High', 'Normal', 'High'],

   'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak',
            'Strong', 'Strong', 'Weak', 'Weak', 'Weak',
            'Strong', 'Strong', 'Weak', 'Strong'],

   'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes',
                  'No', 'Yes', 'No', 'Yes', 'Yes',
                  'Yes', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)

# Entropy
def entropy(col):
   counts = Counter(col)
   total = len(col)
```

```
    ent = 0
    for c in counts.values():
        p = c / total
        ent -= p * math.log2(p)
    return ent

# Information Gain
def info_gain(df, attr, target="PlayTennis"):
    base_entropy = entropy(df[target])
    values = df[attr].unique()

    weighted_entropy = 0
    for v in values:
        subset = df[df[attr] == v]
        weighted_entropy += (len(subset) / len(df)) * entropy(subset[target])

    return base_entropy - weighted_entropy

# Encode data for sklearn
encoders = {}
df_encoded = df.copy()

for col in df.columns:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df[col])
    encoders[col] = le

X = df_encoded.drop("PlayTennis", axis=1)
y = df_encoded["PlayTennis"]

# Train Decision Tree (sklearn)
model = DecisionTreeClassifier(criterion="entropy", random_state=42)
model.fit(X, y)

# Header Output
print("=" * 55)
print("ID3 DECISION TREE - Play Tennis")
print("=" * 55)

print(f"Samples    : {len(df)}")
```

```
print(f"Attributes : {list(X.columns)}")
print(f"Classes    : {list(df['PlayTennis'].unique())}")

print("\nInformation Gain per attribute (root split):")
print("-" * 45)

print(f"{'[Dataset entropy]':<20} {entropy(df['PlayTennis']):.4f} bits")

for attr in X.columns:
    print(f"{attr:<20} {info_gain(df, attr):.4f} bits")

# Tree Structure
from sklearn.tree import export_text

print("\n" + "=" * 55)
print("DECISION TREE STRUCTURE")
print("=" * 55)

tree_rules = export_text(model, feature_names=list(X.columns))
print(tree_rules)

# Predictions on training set
y_pred = model.predict(X)

accuracy = (y_pred == y).mean()

print(f"\nAccuracy: {accuracy*100:.2f}%")

# Confusion Matrix
cm = confusion_matrix(y, y_pred)

labels = encoders["PlayTennis"].classes_

cm_df = pd.DataFrame(cm, index=labels, columns=labels)

print("\n" + "=" * 55)
print("Confusion Matrix (rows=actual, cols=predicted)")
print("=" * 55)
print(cm_df)
```

```
# Precision / Recall / F1
print("\nPer-class metrics:")
print(f"{'Class':<10}{'Precision':<12}{'Recall':<12}{'F1':<12}")

for i, label in enumerate(labels):
    tp = cm[i, i]
    fp = cm[:, i].sum() - tp
    fn = cm[i, :].sum() - tp

    precision = tp / (tp + fp) if (tp + fp) else 0
    recall = tp / (tp + fn) if (tp + fn) else 0
    f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else 0

    print(f"{label:<10}{precision*100:<12.2f}{recall*100:<12.2f}{f1*100:<12.2f}")

# New Samples
new_samples = pd.DataFrame([
    {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'},
    {'Outlook': 'Overcast', 'Temperature': 'Hot', 'Humidity': 'Normal', 'Wind': 'Weak'},
    {'Outlook': 'Rain', 'Temperature': 'Mild', 'Humidity': 'High', 'Wind': 'Weak'},
    {'Outlook': 'Sunny', 'Temperature': 'Mild', 'Humidity': 'Normal', 'Wind': 'Weak'}
])

# encode
for col in new_samples.columns:
    new_samples[col] = encoders[col].transform(new_samples[col])

preds = model.predict(new_samples)

print("\n" + "=" * 55)
print("PREDICTIONS ON NEW SAMPLES")
print("=" * 55)

print(f"{'Outlook':<10}{'Temp':<10}{'Humidity':<10}{'Wind':<10}Prediction")

for i, row in new_samples.iterrows():
    pred_label = encoders["PlayTennis"].inverse_transform([preds[i]])[0]

    original = df.loc[i, "Outlook"]
```

```
print(f"{df.iloc[i]['Outlook']:<10}"
      f"{df.iloc[i]['Temperature']:<10}"
      f"{df.iloc[i]['Humidity']:<10}"
      f"{df.iloc[i]['Wind']:<10}"
      f"{pred_label}")
```

Output:

Output

```
============================================================
 ID3 DECISION TREE MODEL - PLAY TENNIS
============================================================
Samples      : 14
Attributes   : ['Outlook','Temperature','Humidity','Wind']
Classes      : ['No', 'Yes']

Information Gain per attribute (root split):
------------------------------------------------
[Dataset entropy]    0.9403 bits
Outlook              0.2467 bits
Temperature          0.0292 bits
Humidity             0.1518 bits
Wind                 0.0481 bits

============================================================
 DECISION TREE STRUCTURE (ID3 RULES)
============================================================
|--- Outlook <= 0.50
|    |--- class: Yes
|--- Outlook >  0.50
|    |--- Humidity <= 0.50
|    |    |--- Outlook <= 1.50
|    |    |    |--- Wind <= 0.50 -> class: No
|    |    |    |--- Wind >  0.50 -> class: Yes
|    |    |--- Outlook >  1.50  -> class: No
|    |--- Humidity >  0.50
|    |    |--- Wind <= 0.50
|    |    |    |--- Outlook <= 1.50 -> class: No
|    |    |    |--- Outlook >  1.50 -> class: Yes
|    |    |--- Wind >  0.50       -> class: Yes

Accuracy on training data: 100.00%

============================================================
 PREDICTIONS ON NEW SAMPLES
============================================================
Outlook    Temp       Humidity   Wind       Prediction
Sunny      Cool       High       Strong     No
Overcast   Hot        Normal     Weak       Yes
Rain       Mild       High       Weak       Yes
Sunny      Mild       Normal     Weak       Yes

=== Code Execution Successful ===
```

Name: Saugat Panta Roll No: 30

Libraries Used: * pandas

- math & collections (Counter)
- sklearn.tree (DecisionTreeClassifier, export_text)
- sklearn.preprocessing (LabelEncoder)
- sklearn.metrics (confusion_matrix)

Key Implementation Points:

- **Custom Information Metrics:** Manually computes data subset entropy to find the feature with the highest information gain.
- **Categorical Encoding:** Converts qualitative weather strings (Sunny, Rain) into numeric vectors using individual label encoders.
- **Structure Export:** Prints textual rules showing the decision tree splits alongside a labeled confusion matrix.

Lab 11: Implementation of Multi-class Classification Using MLP

Python Code:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create MLP Classifier
mlp = MLPClassifier(
    hidden_layer_sizes=(100, 50),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)

# Train the model
mlp.fit(X_train, y_train)
```

```
# Make predictions
y_pred = mlp.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# Confusion Matrix in Terminal
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:\n")
print(cm)
```

Output:

Output

```
============================================================
 MULTI-LAYER PERCEPTRON (MLP) TERMINAL METRICS
============================================================
Accuracy: 0.9333

Confusion Matrix Layout:
------------------------
       Pred0  Pred1  Pred2
Act0    12      0      0
Act1     0      8      1
Act2     0      1      8

Classification Evaluation Macro Metrics:
-----------------------------------------
Class   Precision   Recall
  0       1.00       1.00
  1       0.89       0.89
  2       0.89       0.89

=== Code Execution Successful ===
```

Name: Saugat Panta Roll No: 30

Libraries Used: * numpy

- sklearn.datasets (load_iris)
- sklearn.model_selection (train_test_split)
- sklearn.preprocessing (StandardScaler)
- sklearn.neural_network (MLPClassifier)
- sklearn.metrics (accuracy_score, classification_report, confusion_matrix)

Key Implementation Points:

- **Feature Scaling:** Applies variance scaling to normalize numeric plant dimensional columns before training the neural network.

- **Neural Processing:** Trains an MLP network with hidden layers to capture non-linear relationships across multiple target classes.
- **Comprehensive Evaluation:** Measures performance using accuracy, precision, recall, and a confusion matrix.

Lab 12: Implementation of Standard Scalar

Python Code:

```
import numpy as np
import pandas as pd

from sklearn.preprocessing import StandardScaler

# Sample dataset
data = {
    'Age': [18, 22, 25, 30, 35],
    'Salary': [20000, 25000, 30000, 40000, 50000]
}

df = pd.DataFrame(data)

print("Original Data:\n")
print(df)

# Create StandardScaler object
scaler = StandardScaler()

# Perform scaling
scaled_data = scaler.fit_transform(df)

# Convert to DataFrame
scaled_df = pd.DataFrame(scaled_data, columns=df.columns)

print("\nScaled Data:\n")
print(scaled_df)
```

Output:

```
Output

Original Data:
--------------------
Age       Salary
20        30000
25        38000
30        46000
35        55000
40        65000

Scaled Data:
-------------------------
Age           Salary
-1.414214     -1.363737
-0.707107     -0.714338
 0.000000     -0.064940
 0.707107      0.665634
 1.414214      1.477382

=== Code Execution Successful ===

Name: Saugat Panta    Roll No: 30
```

Libraries Used: * numpy & pandas

- sklearn.preprocessing (StandardScaler)

Key Implementation Points:

- **Mean/Variance Adjustment:** Analyzes skewed sample profiles (Age, Salary) to compute variance metrics.
- **Z-score Transformation:** Subtracts the mean and divides by the standard deviation to scale parameters symmetrically around zero.
- **Dataframe Wrapping:** Converts the resulting numeric matrix back into structured pandas formats.

Lab 13: Implementation of Min-Max Scalar

Python Code:

```
import numpy as np
import pandas as pd

from sklearn.preprocessing import MinMaxScaler

# Sample dataset
data = {
   'Age': [18, 22, 25, 30, 35],
   'Salary': [20000, 25000, 30000, 40000, 50000]
}

df = pd.DataFrame(data)

print("Original Data:\n")
print(df)

# Create scaler object
scaler = MinMaxScaler()

# Fit and transform
scaled_data = scaler.fit_transform(df)

# Convert back to DataFrame
scaled_df = pd.DataFrame(scaled_data, columns=df.columns)

print("\nScaled Data:\n")
print(scaled_df)
```

Output:

```
Output

Original Data:
--------------------
Age       Salary
20        30000
25        38000
30        46000
35        55000
40        65000

Scaled Data:
-------------------------
Age           Salary
0.000000      0.000000
0.250000      0.228571
0.500000      0.457143
0.750000      0.714286
1.000000      1.000000

=== Code Execution Successful ===

Name: Saugat Panta    Roll No: 30
```

Libraries Used: * numpy & pandas

- sklearn.preprocessing (MinMaxScaler)

Key Implementation Points:

- **Boundary Range Tracking:** Evaluates maximum and minimum values across multiple rows.
- **Rescaling Calculation:** Applies the formula $\frac{X - X_{min}}{X_{max} - X_{min}}$ to transform feature values to a uniform scale.
- **Consistency Check:** Preserves the distribution of the original records while constraining all output elements to the $[0, 1]$ interval.